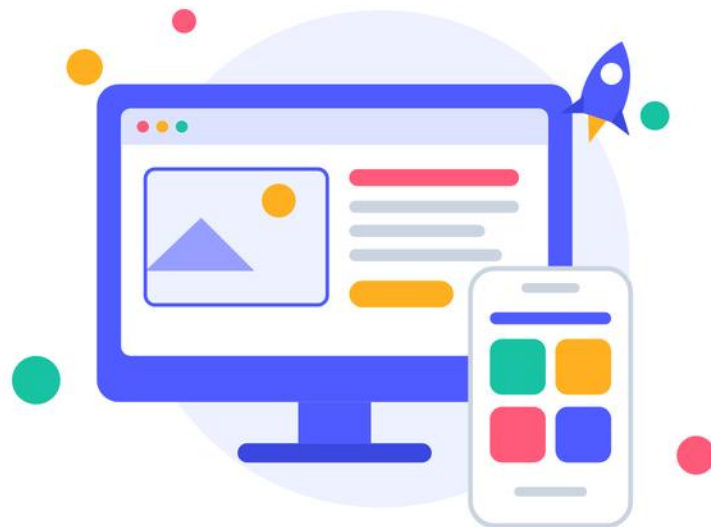


WEEK 02 · CASCADE · BOX MODEL · TOKENS

MAKING IT LOOK LIKE A DECISION

Last week you built a skeleton. Today it stops looking like a document and starts looking like something you would show someone.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN



Without CSS, every website would look plain, like a Word document.

FROM THIS COURSE, BEFORE YOU WERE IN IT

WHERE WE ARE

Week 2 of 12. You have a valid, accessible page and a repo with history in it.

AFTER TODAY YOU CAN

Write a stylesheet from an empty file, work out which rule wins and why, and build a colour and spacing system instead of one-off values.

WHAT THIS UNLOCKS

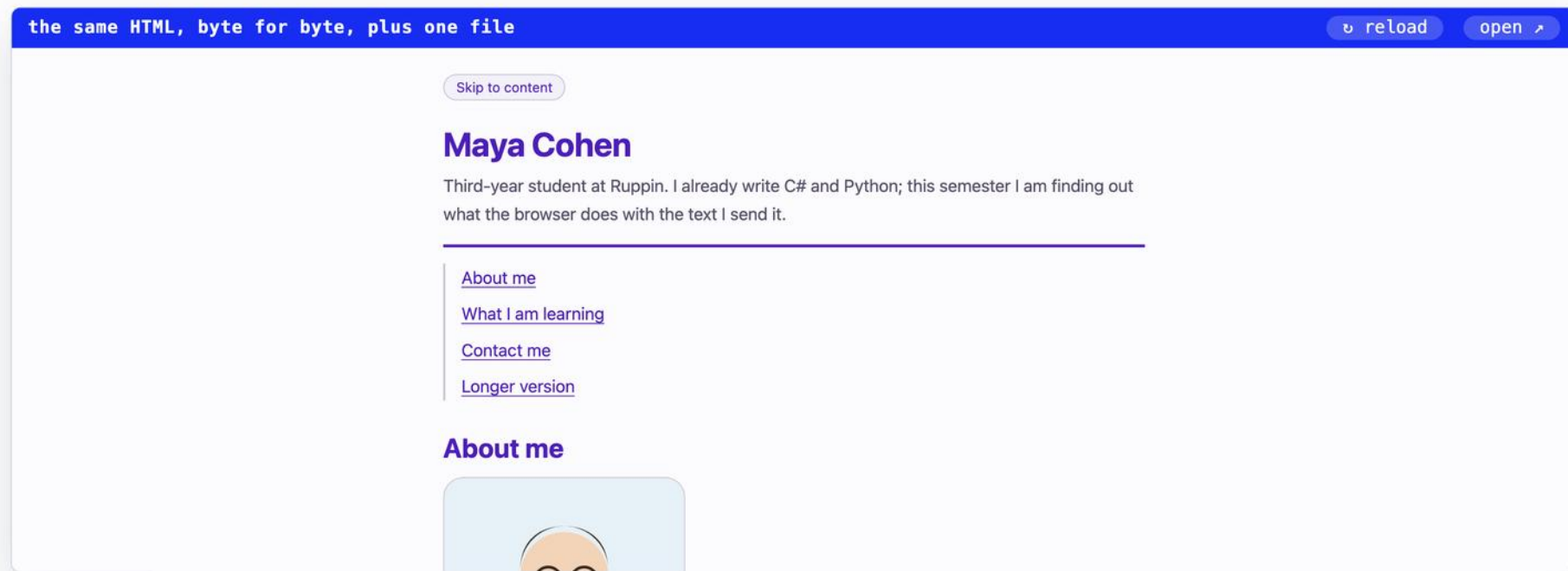
Weeks 3, 4 and 5 are all layout on top of this. Tailwind in week 5 is these exact properties with shorter names.

THIS IS YOUR PAGE. RIGHT NOW.



► **SAY IT OUT LOUD** — this page is **CORRECT**. Valid, accessible, semantic. And nobody would look at it twice.

THIS IS YOUR PAGE AT 16:30 TODAY.



► **THE POINT** — the markup did not change. Not one element. The whole difference is `styles.css`, and by the end of the lab it is yours.

A STYLE SHEET IS A LIST OF RULES

selector `h1` — which elements this applies to.

declaration block everything between the braces.

declaration one `property: value;` pair. Three of them here.

That is the entire syntax. Everything difficult about CSS is about which rule wins and what a value means — never about the punctuation.

```
1 h1 {  
2   color: hsl(258 72% 42%);  
3   font-size: 2rem;  
4   line-height: 1.2;  
5 }
```

ACT 1 · THREE WAYS, ONE ANSWER

THREE WAYS TO ATTACH CSS

```
1  <!-- 1. inline: on the element itself -->
2  <h1 style="color: teal">Ruppín Coffee Club</h1>
3
4  <!-- 2. internal: a <style> block in the head -->
5  <head>
6    <style>
7      h1 { color: teal; }
8    </style>
9  </head>
10
11 <!-- 3. external: a separate file, linked -->
12 <link rel="stylesheet" href="styles.css" />
```

All three render identically. Only one of them survives contact with a second page.



CHANGE THE BRAND COLOUR. COUNT THE EDITS.

demos/01-three-ways/01-inline — every decision, eight times

↻ reload

open ↗

Ruppín Coffee Club

Third floor, next to the printer that never works.

This week's beans

Ethiopia, Guji. Tastes of stone fruit, and of someone else paying.

Opening hours

Run by second-years who should have been revising.

► **HANDS-ON** demos/01-three-ways/ — Change teal to red in 01-inline/. Count out loud. Then do the same in 03-external/.

WHY EXTERNAL, IN THREE SENTENCES

ONE FILE, MANY PAGES

Every page points at the same stylesheet. A change lands everywhere at once, and two pages cannot drift apart.

THE BROWSER CACHES IT

It is downloaded once for the whole site, not re-sent inside every page.

STRUCTURE STAYS STRUCTURE

The markup goes back to saying what things ARE. That is the file you spent last week getting right.

External CSS lets you use one stylesheet for several different pages.

ONE THING YOU MAY HAVE BEEN TOLD, THAT IS WRONG

WRONG

"Specificity goes: **inline > internal > external**."

It appears in a lot of course material, including an older version of this one.

RIGHT

Where a rule lives is not specificity at all. An internal rule and an external rule with the same selector are decided by **source order** — whichever the browser reads last.

Move the `<link>` below the `<style>` block and the answer flips.

Nothing about "internal" made it stronger.

ACT 2 · THE PROBLEM

"I CHANGED THE CSS AND NOTHING HAPPENED"

demos/02-cascade/01-conflict - three rules, all losing

↺ reload

open ↗

This heading is supposed to be teal

This callout is supposed to be blue. Someone edited it. Twice.

Everything in this box is red, because the box says so.

Everything except this button:

Order a flat white

► **HANDS-ON** demos/02-cascade/01-conflict/ — Three symptoms. Do not explain them. Ask which one they would debug first.

THE CASCADE IS AN ALGORITHM. HERE IT IS.

1 Origin

browser default, then yours. Yours wins.

2 Importance

!important jumps the queue. Almost never the answer.

3 Specificity

a triple — (a, b, c) — compared column by column.

4 Source order

last one written wins. Only reached on an exact tie.

There is no fifth step. If two rules are still tied after source order, they are the same rule.



SPECIFICITY IS NOT A COUNT

HOW EVERYONE LEARNS IT

"More selectors means stronger. Count them."

`.site .nav .list .item a` — five pieces, so surely it beats `#cta`, which is one.

WHAT IT ACTUALLY IS

Three numbers, compared like a version number.

`#cta` is `(1, 0, 0)`.

`.site .nav .list .item a` is `(0, 4, 1)`.

1.0.0 beats 0.4.1. It is not close, and it never will be.

COMPUTING THE TRIPLE

- a** `#id`
- b** `.class` · `[attribute]` · `:pseudo-class`
- c** element names · `::pseudo-element`

`*`, combinators and `:where()` count nothing. `:is()`, `:not()` and `:has()` take the specificity of their strongest argument.

```
1 a /* (0,0,1) */
2 nav a /* (0,0,2) */
3 a.button /* (0,1,1) */
4 a:hover /* (0,1,1) */
5 #cta /* (1,0,0) */
```

`a.button` and `a:hover` tie. That tie is why hover states so often "stop working".



COMPUTE IT YOURSELF

[demos/03-specificity](#) — type any two selectors [↶ reload](#) [open ↷](#)

SPECIFICITY CALCULATOR

[one id vs five selectors](#) [four elements vs one class](#) [a tie — so order decides](#) [a pseudo-element earns c](#) [:where\(\) is free](#) [an attribute weighs like a pseudo-class](#)

Selector A

#cta

(1 , 0 , 0) 1 selector counted

[a #cta](#)

Selector B

.site .nav .list .item a

(0 , 4 , 1) 5 selectors counted

[b .site](#) [b .nav](#) [b .list](#) [b .item](#) [c a](#)

A wins: (1,0,0) beats (0,4,1) — even though B has more pieces in it.

► **HANDS-ON** [demos/03-specificity/](#) — Six presets, every one a case where counting gives the wrong answer. Vote before pressing.

WHAT TO DO WHEN YOUR RULE LOSES

THE REFLEX

```
.title {  
  color: teal !important;  
}
```

Works in three seconds. Costs you for the rest of the project.

THE FIX

```
/* was: #page-title { color: grey } */  
.page-title {  
  color: grey;  
}
```

Lower the winner instead of raising the loser. Now everything is a class again, and everything is overridable.

THE CASCADE, SHOWN ON THE PAGE

`demos/02-cascade/02-explained` — click an element, pick a property ↶ reload open ↷

This heading is supposed to be teal

This callout is supposed to be blue. Someone edited it. Twice.

Everything in this box is red, because the box says so.

Everything except this button:

Order a flat white

CASCADE INSPECTOR

Click any element above. Then pick a property.

Property

► **HANDS-ON** `demos/02-cascade/02-explained/` — Click the heading with `color` selected. Two rows, sorted the way the browser sorts them.

INHERITANCE: SOME THINGS FALL DOWN THE TREE

INHERITS

color font-family font-size
line-height text-align
visibility

Anything about TEXT. Set it once on `body` and the page is styled.

DOES NOT INHERIT

margin padding border
background width display

Anything about the BOX. Imagine if padding inherited — every nested element would nest its own padding.

THE EXCEPTION THAT BITES

Form controls — `input`, `button`, `textarea` — have their OWN font and colour from the browser, and a set value beats an inherited one.

`font: inherit` is how you opt back in.

THREE KEYWORDS WORTH KNOWING

```
1  /* take the parent's value, even for a property that does not inherit */
2  .card p { margin: inherit; }
3
4  /* back to the CSS specification's default, ignoring the browser's stylesheet */
5  button { all: initial; }
6
7  /* inherit if the property inherits, otherwise initial */
8  .reset { border: unset; }
```

`inherit` is the useful one this week. The other two matter when you fight a framework — week 5.



ACT 3 · CHOOSING WHAT TO STYLE

TYPE, CLASS, ID

```
1  /* type: every one of them, everywhere */
2  p { line-height: 1.6; }
3
4  /* class: the ones you marked. Reusable. */
5  .lede { font-size: 1.33rem; }
6
7  /* id: exactly one element on the page */
8  #page-title { color: grey; }
```

Type selectors are the default, and last week's work is what makes them enough.

header, nav, table caption, form fieldset — all reachable by what they ARE.

Use a class when there is no element that means it. "The opening paragraph" has no tag. That is what .lede is for.

Never style by id. It cannot be reused, and at (1,0,0) nothing you write later can override it.

COMBINATORS: RELATIONSHIPS BETWEEN ELEMENTS

```
1  /* descendant: any p anywhere inside nav */
2  nav p { margin: 0; }
3
4  /* child: only a direct child */
5  nav > ul { padding-left: 0; }
6
7  /* adjacent sibling: the p immediately after an h2 */
8  h2 + p { font-size: 1.2rem; }
9
10 /* general sibling: every p after an h2, same parent */
11 h2 ~ p { color: grey; }
```

The space is a combinator too, and it is the one people forget: `nav p` and `nav.p` are completely different selectors.

ATTRIBUTES, STATES, AND THINGS THAT ARE NOT THERE

```
1 /* attribute: no class needed */
2 input[type='email'] { ... }
3
4 /* state: :hover, :focus-visible, :required */
5 a:hover { ... }
6 input:required { border-left-width: 4px; }
7
8 /* position among siblings */
9 tbody tr:nth-child(even) { ... }
```

`:focus-visible` rather than `:focus`: the browser decides whether the user is navigating by keyboard, and it is better at that than you are. A focus ring on every mouse click looks like a bug.

`::before` and `::after` create a box that is not in the HTML. They need a `content` property to exist at all — even `content: ''`.



GROUPING, AND HOW FLAT SHOULD A SELECTOR BE

TOO DEEP

```
body main section
  article div p a {
    color: teal;
  }
```

Fragile: one new wrapper and it stops matching. And at `(0,0,7)` it is hard to override.

FLAT

```
h1,
h2,
h3 {
  line-height: 1.18;
}

a { color: teal; }
```

Says what it means, applies everywhere, easy to override on purpose.

A long chain is almost always somebody raising specificity until something won, instead of finding out what was winning.

ACT 4 · EVERYTHING IS A BOX

FOUR RINGS, AND ONLY ONE OF THEM IS YOURS

content

the text or the image itself.

padding

space INSIDE the border. Takes the background.

border

the edge. Has width, style and colour.

margin

space OUTSIDE the border. Always transparent.

Padding and margin look identical on a white page and behave completely differently the moment there is a background colour or a border.

Padding is part of the box. Margin is the space around it. That sentence answers most of the questions this section produces.

LIVE CODING · BREAK 1

THE BOX, MEASURED BY THE BROWSER

demos/04-box-model/03-visualiser — nothing here is simulated ↶ reload open ↷

THE DECLARATION

box-sizing
content-box (the default) ▾

width 200px

padding 20px

border-width 4px

margin 16px

content padding border
 margin

THE BOX, IN A 300PX RAIL

```
box-sizing: content-box
```

width is the CONTENT only, so padding and border are ADDED:

content	200	
+ padding ×2	40	
+ border ×2	8	
= box on page	248	(you asked for 200)
+ margin ×2	32	
= space taken	280	(margin is outside the box, always)

Fits the 300px rail, with 16px to spare.

► **HANDS-ON** [demos/04-box-model/03-visualiser/](#) — width 200, padding 20, border 4. Ask how wide the box is BEFORE reading the number.

TWO BOXES THAT WERE TOLD TO BE 240 WIDE

demos/04-box-model/01-overflow — both escape a 260px container

↻ reload

open ↗

TWO BOXES THAT WERE ASKED TO BE 240PX WIDE

The dashed containers are 260px. Both children were given width: 240px.

Ethiopia, Guji

Stone fruit. Someone else paying.

Email

you@ruppin.ac.il

measure them

Guess first. Then press the button.

► **HANDS-ON** demos/04-box-model/01-overflow/ — Press measure them. Neither is 240.

THREE LINES, AT THE TOP OF EVERY STYLESHEET

THE DEFAULT

```
.card {  
  width: 240px;  
  padding: 20px;  
  border: 2px solid;  
}  
/* on the page: 284px */
```

`width` means the CONTENT. Padding and border are added on.

BORDER-BOX

```
*,  
*::before,  
*::after {  
  box-sizing: border-box;  
}  
/* on the page: 240px */
```

`width` means the whole box. Which is what everyone thought it meant.

MARGINS COLLAPSE. THIS IS NOT A BUG.

```
1 h2 { margin-bottom: 32px; }  
2  
3 p { margin-top: 16px; }  
4  
5 /* the gap between them: 32px, not 48px */
```

Two vertical margins that meet do not add up — the larger one wins. It only happens **vertically**, and only between margins with nothing between them: no border, no padding, no content.

It is what stops a document from accumulating enormous gaps between every heading and paragraph. Once you know it exists it stops being mysterious.

NOT EVERY BOX TAKES A WIDTH

BLOCK BOXES

`div` `p` `h1` `section` `ul` `li`

Start on a new line, take the full width available, accept `width` and `height`.

INLINE BOXES

`a` `span` `strong` `em` `code` `label`

Sit inside a line of text. `width` and `height` do nothing to them, and vertical padding does not push neighbours away.

"I set a width on my link and nothing happened" is not a bug. Next week's `display` is the property that changes which kind of box an element makes.

THE FOUR SHORTHANDS YOU WILL ACTUALLY TYPE

```
1  /* one value: all four sides */
2  padding: 16px;
3
4  /* two: vertical, then horizontal */
5  padding: 16px 24px;
6
7  /* four: clockwise from the top – top, right, bottom, left */
8  padding: 8px 16px 24px 16px;
9
10 /* border and radius follow the same idea */
11 border: 1px solid hsl(220 20% 80%);
```

Clockwise from the top. TRouBLE, if a mnemonic helps.



ACT 5 · VALUES

ONE COLOUR, THREE NOTATIONS

```
1 /* hex: two hex digits per channel. Compact, unreadable. */  
2 color: #4c1eb8;  
3  
4 /* rgb: how much of each light. Readable, still not meaningful. */  
5 color: rgb(76 30 184);  
6  
7 /* hsl: hue, saturation, lightness. Meaningful. */  
8 color: hsl(258 72% 42%);
```

Identical pixels. Now make it 10% lighter — and notice that only one of the three lets you do that without a colour picker.

A WHOLE PALETTE FROM ONE NUMBER

demos/06-colour/hsl-explorer – drag the hue across the range

reload

open

ONE COLOUR, THREE NUMBERS

hue 234



saturation 89%



lightness 52%



■ hsl hsl(234 89% 52%)
rgb rgb(24 45 242)
hex #182df2

THE WHOLE PALETTE, DERIVED FROM THAT ONE HUE

Every row is the same hue. Only saturation and lightness move.

	token	hsl	hex	on surface
	--color-surface	hsl(234 40% 98%)	#f8f8fc	1.00:1 – this is the surface
	--color-border	hsl(234 30% 82%)	#c3c6df	1.59:1 – decorative – no minimum
	--color-border-strong	hsl(234 34% 48%)	#5159a4	5.99:1 AA
	--color-accent	hsl(234 89% 52%)	#182df2	7.36:1 AA
	--color-accent-strong	hsl(234 89% 38%)	#0b1cb7	10.65:1 AA
	--color-text-muted	hsl(234 20% 34%)	#454968	8.26:1 AA
	--color-text	hsl(234 47% 11%)	#0f1229	17.48:1 AA

Drag the hue towards yellow and watch the accent row go red. Same lightness, same saturation, failing contrast — because HSL's "lightness" is a number in a colour model, not how bright the colour looks to an eye. Yellow at 52% is far brighter than blue at 52%. This is why the ratio is checked and never assumed.

Ruppin Coffee Club

Body text on the surface colour, at the ratio in the table.

[A link, in the accent colour](#)

► **HANDS-ON** demos/06-colour/hsl-explorer/ — Drag HUE end to end. The palette stays coherent at every stop. Then ask them to do that with hex.

CONTRAST IS A NUMBER, AND YOU CAN CHECK IT

4.5:1

body text. The line most designs fail.

3:1

large text — 24px, or 18.66px bold — and meaningful UI edges.

7:1

AAA. Nice, rarely required.

Two colours in, one ratio out. Thirty seconds a pair, and it is the difference between a page that looks nice on your laptop and a page that can be read on a projector, in sunlight, or by someone with low vision.

Tools worth keeping:

webaim.org/resources/contrastchecker ·

color.adobe.com · colorhunt.co

THE TRAP INSIDE HSL

SAME LIGHTNESS, DIFFERENT BRIGHTNESS

`hsl(240 90% 52%)` — blue — passes AA on white.

`hsl(60 90% 52%)` — yellow — fails badly.

Both say 52%.

WHY

"Lightness" in HSL is a coordinate in a colour model, not how bright a colour looks to an eye. Human vision is far more sensitive to green and yellow than to blue.

So: never assume. Check.

FONT STACKS: A LIST, IN ORDER OF PREFERENCE

```
1 font-family:
2   system-ui, -apple-system, 'Segoe UI', Roboto,
3   'Helvetica Neue', Arial, sans-serif;
4
5 /* the last name is the generic family – always end with one */
6 font-family: Georgia, 'Times New Roman', serif;
```

`system-ui` is whatever the reader's own operating system uses. Already installed, already tuned for their screen, downloads nothing.

WEB FONTS, AND WHAT THEY COST

```
1 @font-face {  
2   font-family: 'Heebo';  
3   src: url('/fonts/heebo.woff2') format('woff2');  
4   font-weight: 400;  
5   font-display: swap;  
6 }  
7  
8 /* now it is just another name in the stack */  
9 body { font-family: 'Heebo', system-ui, sans-serif; }
```

Host the file yourself. A font from someone else's CDN is a request to a third party on every page load, a dependency on their uptime, and nothing at all in an exam room with no network.

TWO NUMBERS THAT DECIDE WHETHER TEXT IS READABLE

LINE-HEIGHT

`1.5` to `1.7` for body text. Unitless, so it multiplies the element's own size and keeps working when the size changes.

Headings want `1.1` to `1.2` — at 2rem, body leading opens a canyon between two lines of one heading.

MEASURE

`max-width: 65ch`. One `ch` is the width of a `0` in the current font, so this says "about 65 characters".

A line running the full width of a wide monitor is genuinely hard to read: the eye loses its place coming back.

A TYPE SCALE IS MULTIPLICATION, NOT OPINION

```
1  /* base 17px, each step x1.25 */
2  --font-size-base: 1.0625rem;
3
4  --font-size-sm:    0.85rem;    /* base / 1.25 */
5  --font-size-lg:    1.33rem;    /* base x 1.25 */
6  --font-size-xl:    1.66rem;    /* x 1.25 again */
7  --font-size-2xl:   2.07rem;    /* and again    */
```

Four sizes chosen by one ratio look related. Four sizes chosen one at a time look like four decisions, because they are.

EM COMPOUNDS. REM DOES NOT.

demos/05-units — one rule, five nesting levels

reloadopen

ONE RULE: LI { FONT-SIZE: 0.9EM }

Drinks

Coffee

Filter

Ethiopia

Guji, washed

measure every level

Guess the size of the deepest item first.

► **HANDS-ON** [demos/05-units/](#) — Ask for the size of the deepest item. Press. Then open [02-rem/](#) and drag the reader's font-size slider.

A RULE FOR EVERY UNIT

REM

Type sizes and spacing. Your default for almost everything. Follows the reader's own font-size setting.

EM

Only when a value should scale with THIS element's text — padding inside a button that comes in three sizes.

%

Widths, relative to the containing block. `width: 100%` is the one you will use today.

CH

Measure, and nothing else.
`max-width: 65ch.`

PX

Hairlines and edges: borders, outlines, small radii. Things that should NOT scale.

VW / VH

Viewport-sized things only.
`font-size: 4vw` is unreadable on a phone and absurd on a monitor. Sparingly.

ACT 6 · NAMING THE DECISIONS

"THE CLIENT WANTS IT PURPLE"

demos/07-tokens/01-hardcoded – the same teal, six times

↺ reload

open ↗

Ruppín Coffee Club

Third floor, next to the printer that never works.

This week

Two coffees on the bar, both washed, both from producers we can actually name.

Ethiopia, Guji

Stone fruit, jasmine, and the faint taste of someone else paying.

Colombia, Huila

Caramel and red apple. The safe one, and there is no shame in that.

Joining

Turn up. Put your name on the rota if you want a key.

Run by second-years who should have been revising.

► **HANDS-ON** demos/07-tokens/01-hardcoded/ — Change it to purple live. Count the misses. Then: "and a bit more air between the sections."

CUSTOM PROPERTIES

```
1  :root {
2    --color-accent: hsl(258 72% 42%);
3    --space-4: 1rem;
4  }
5
6  a {
7    color: var(--color-accent);
8    padding: var(--space-4);
9  }
10
11 /* a fallback, if the token was never defined */
12 color: var(--color-accent, hotpink);
```

Two dashes to declare, `var()` to read. That is the whole feature.

FOUR FAMILIES. THAT IS THE WHOLE SYSTEM.

`--color--*`

bg · surface · border · text
· text-muted · accent

`--space--*`

a scale in rem: 1 · 2 · 3 · 4 ·
5 · 6. Steps, never one-off
numbers.

`--radius--*`

sm · md · lg · full. Two is
enough to start.

`--font--*`

the stack, plus the size
scale from act 5.

Name them for the JOB, never for the value. `--color-accent`, not
`--color-purple` — because the day the accent stops being purple, a
stylesheet full of `--color-purple: green` is unreadable.

THE PAYOFF, AND IT IS BIGGER THAN COLOUR

demos/07-tokens/03-themed — press the toggle, then read the stylesheet

↻ reload

open ↗

switch to dark

Ruppín Coffee Club

Third floor, next to the printer that never works.

This week

Two coffees on the bar, both washed, both from producers we can actually name.

Ethiopia, Guji

Stone fruit, jasmine, and the faint taste of someone else paying.

Colombia, Huila

Caramel and red apple. The safe one, and there is no shame in that.

Joining

Turn up. [Put your name on the rota](#) if you want a key.

Run by second-years who should have been revising.

► **HANDS-ON** demos/07-tokens/03-themed/ — Count the rules that changed below the token block. It is zero.

A TOKEN CAN HOLD ANYTHING, INCLUDING TIME

```
1  :root {  
2    --duration: 150ms;  
3  }  
4  
5  a { transition: color var(--duration) ease; }  
6  
7  /* some people get motion sickness from moving interfaces */  
8  @media (prefers-reduced-motion: reduce) {  
9    :root {  
10     --duration: 0s;  
11   }  
12 }
```

The usual version of this forces `transition: none !important` on every element. With a token it is one value, and no override war.

YOU HAVE BEEN LOOKING AT ONE ALL MORNING

```
1 /* shared/tokens.css */  
2 --color-accent: hsl(234 89% 52%);  
3 --space-4: 1rem;  
4 --radius-xl: 28px;  
5 --font-display: 'Heebo', system-ui;  
6 --duration-normal: 220ms;
```

These slides, every demo you have seen today, and every Hebrew PDF in this course are drawn from one file with about forty tokens in it.

It is the same idea you are about to write, at the scale of a whole course. Open it — `shared/tokens.css` in the materials repo — and read the comments.

THREE QUESTIONS BEFORE YOU LEAVE

1. `#cta` against `.site .nav .list .item a`. Which wins, and what are the two triples?
2. An element has `width: 300px`, `padding: 20px` and a `2px` border. How wide is it on the page? Now add the three lines and answer again.
3. You want the whole page one hue cooler. In a stylesheet with tokens, how many lines do you change? In one without?

WEEK 02 · NEXT

THE LAB, AND WHAT IS DUE

IN THE LAB, NOW

Your own week-1 page in `week-02/`, plus `styles.css`: the reset, a full token set, a type scale, a colour system and hover and focus states.

No Flexbox, no Grid, no positioning.

ALSO PUBLISHED TODAY

Home assignment 1 — a three-page mini-site sharing one stylesheet. Due before week 4, and it is the first of the four that make up 10% of the grade.

BEFORE NEXT WEEK

Nothing to install. Next week the boxes stop stacking and start lining up: `display`, `position`, and Flexbox.